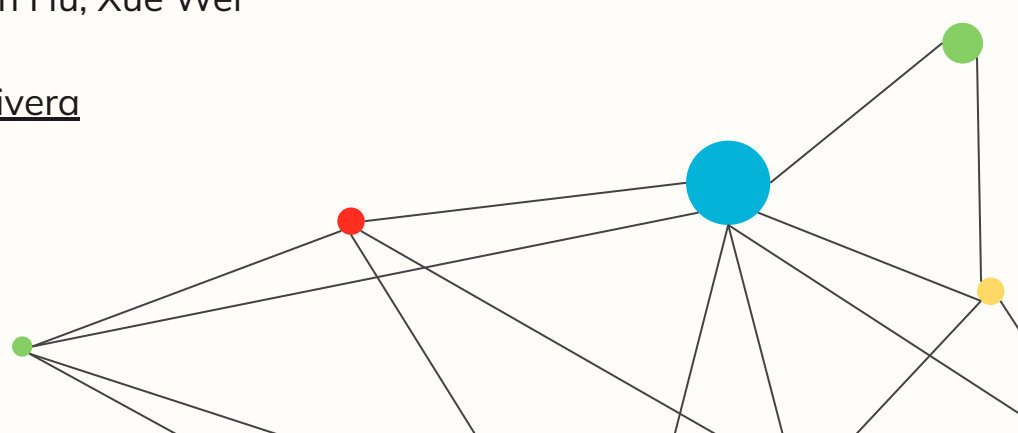
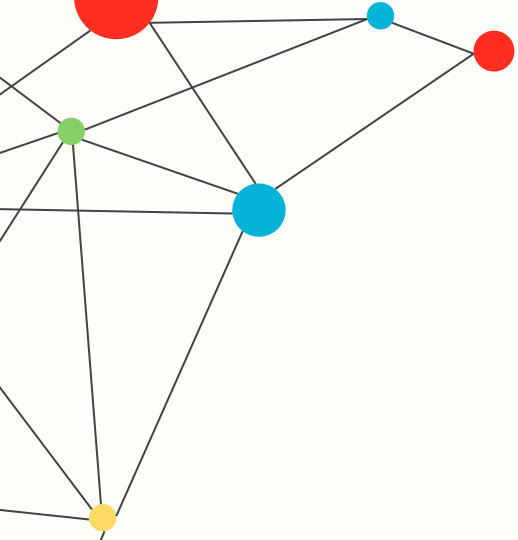




Enabling High Quality Real-Time Communications with Adaptive Frame-Rate

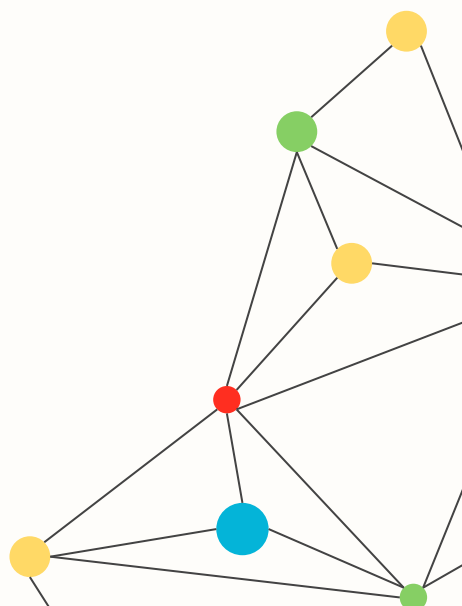
Zili Meng, Tingfeng Wang, Yixin Shen, Bo Wang and
Mingwei Xu, Rui Han and Honghao Liu, Venkat Arun,
Hongxin Hu, Xue Wei

Presented by: Carson Rivera



01

Background

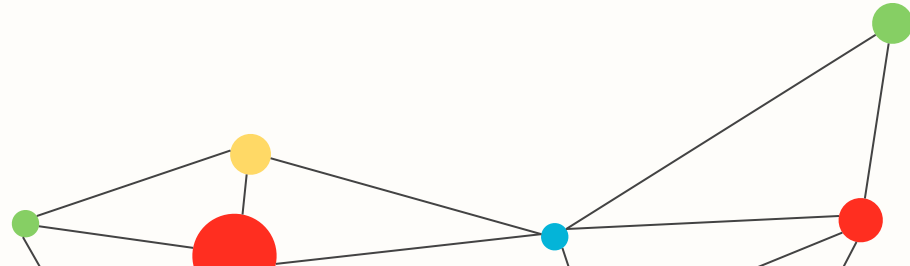
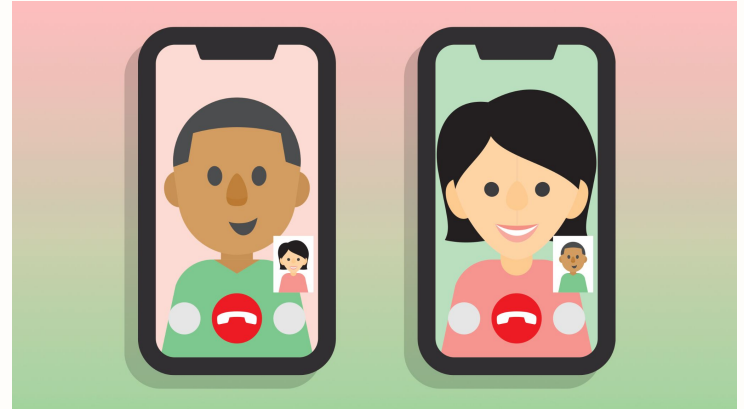




Real-time Communication

Real-time communication (RTC) is the instant exchange of voice, video, or data between people or systems with **minimal latency**.

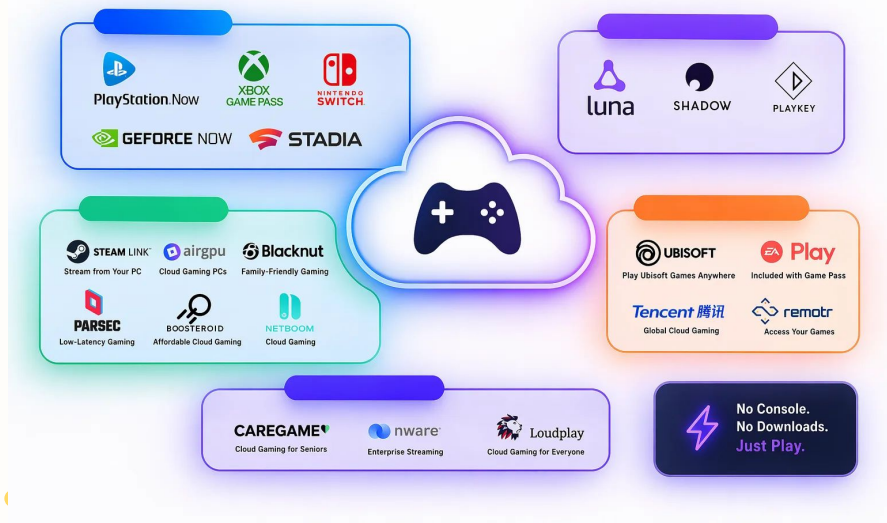
There are **two** types of RTC systems: hard real-time (HRT) and soft real-time (SRT).

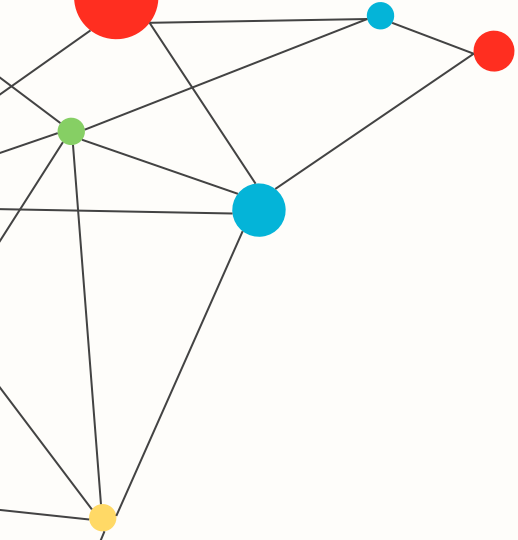


RTC is Growing in Popularity

Customers for applications now **expect** RTC. Systems now have to focus on both accuracy and live communication.

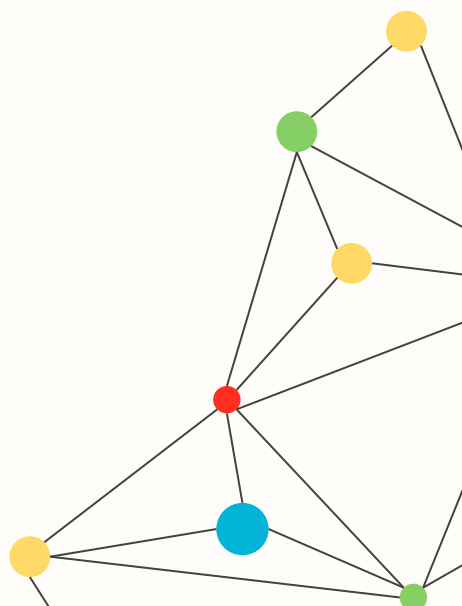
Some examples include: video conferencing, remote surgery, teleoperation of robots, and **cloud video gaming**.





02

Motivation

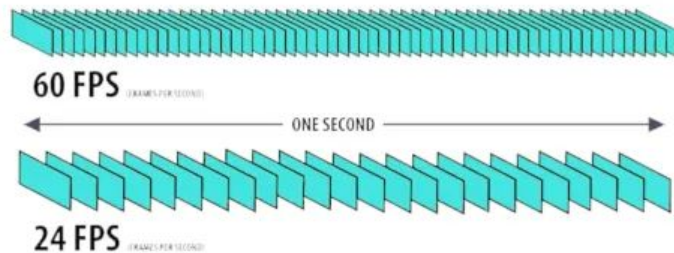


Quality of RTC is Increasing

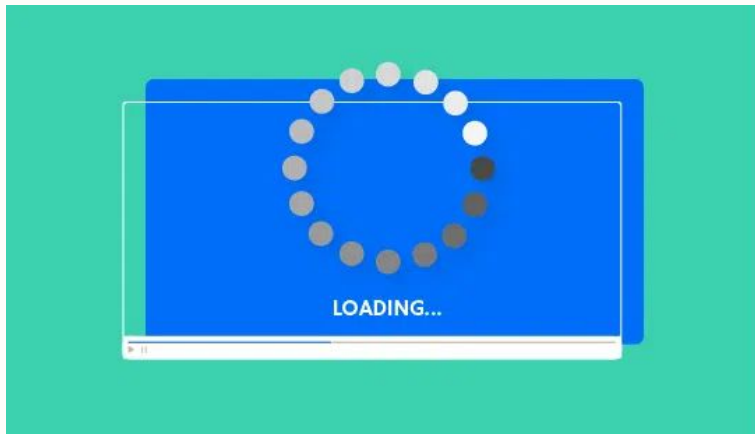
The quality of RTC applications are improving in both **frame rate** and **resolution**.

Legacy applications were at 30 or fewer frames per second (fps) and 240-720p while **new systems** aim for 1080p+ and 60-120fps.

Frames Per Second: FPS



RTC Applications Want Low Stall Ratios



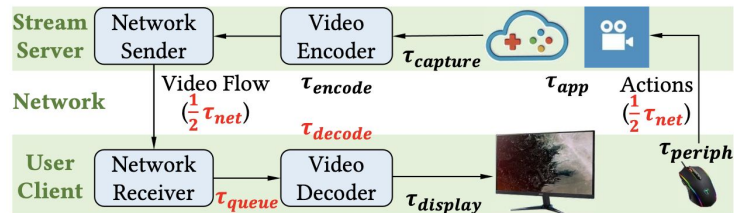
Stall ratio is the time spent **frozen/buffering** divided by the total time streaming.

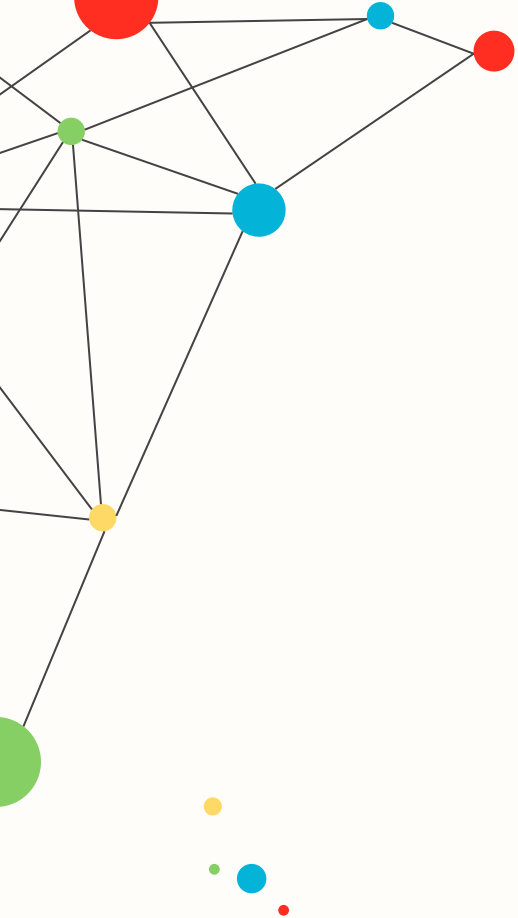
In high-pressure situations, even a stall ratio of 0.1% can mean **life or death**.

Increase in Quality Leads to Overload

In an RTC system, there is a decoder queue which acts as the **buffer** between the network stack and media decoder.

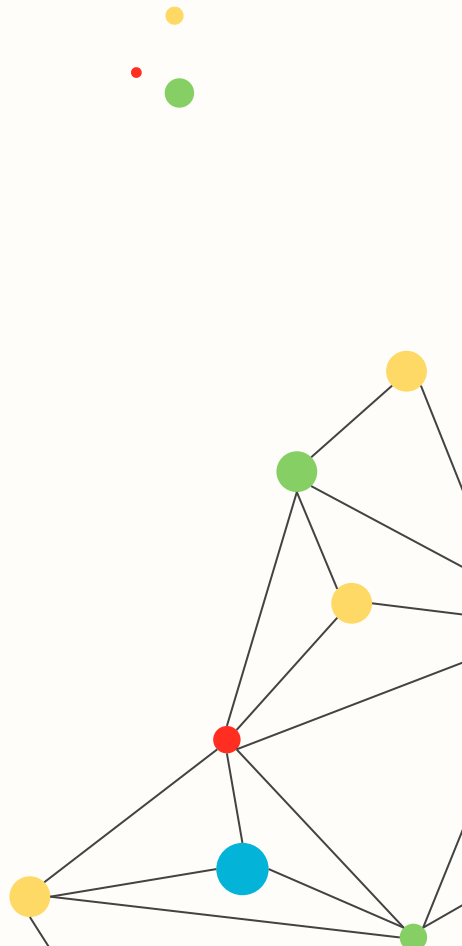
In legacy systems, packets departed from the decoder queue at fast rates while in modern systems, the **arrival rate of packets outpaces the departure rate**.





03

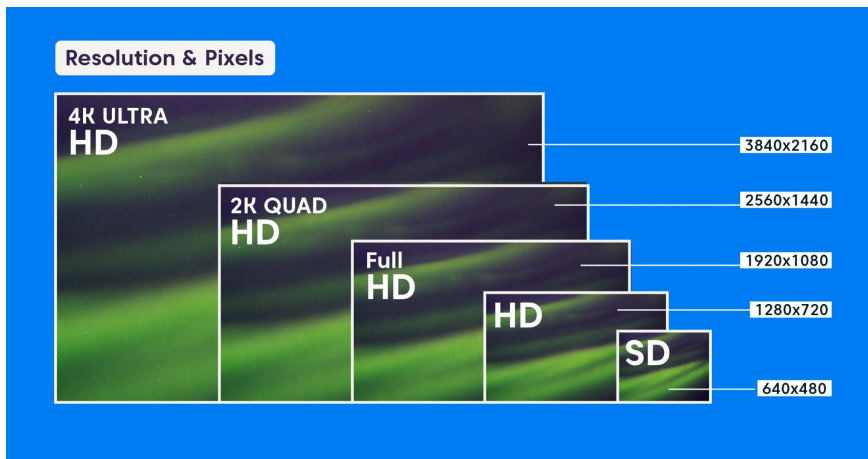
Design



Adaptive Frame Rate

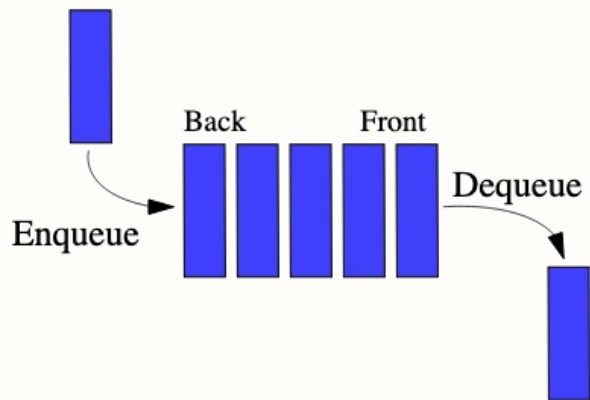
Rather than **adapting the resolution** of video like previous works, this paper proposes dynamically **adapting the frame rate**.

If the server sends 60 frames and your device can process 40 frames, the queue overloads. We can instead choose to **send fewer frames** from the server.



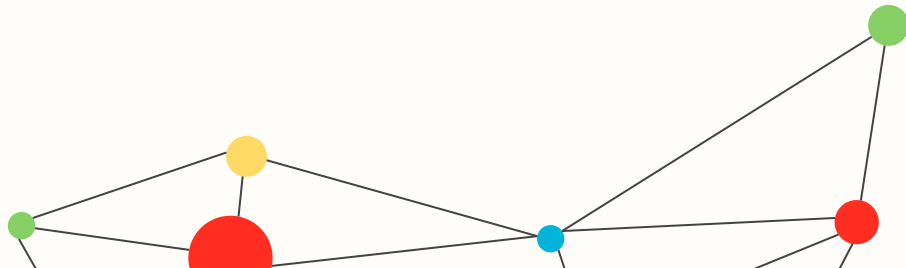


Managing the Queue



Prior papers using queue management focus on controlling the **queue length**.

If the queue length is under a certain threshold, signal to increase sending rate. If the length is over, signal to decrease sending rate.



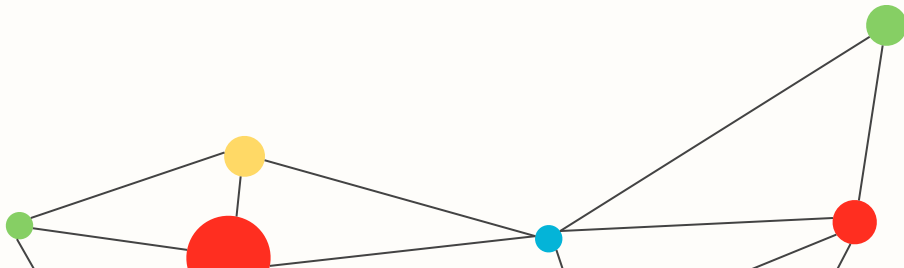


Frame Delay Problem

This paper aims to achieve an ultra-low queueing delay.

A queue of video frames instead of packets leads to a **base delay** of 10ms even at size 1.

Instead of using the queue information, use the information from the **departure and arrival** processes.





Design Concerns

Decoder Degradation

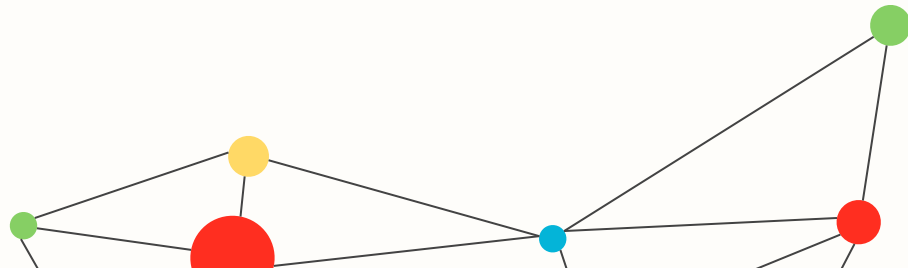
If the user's device is too hot, the client decoder cannot process frames at the desired frame rate.

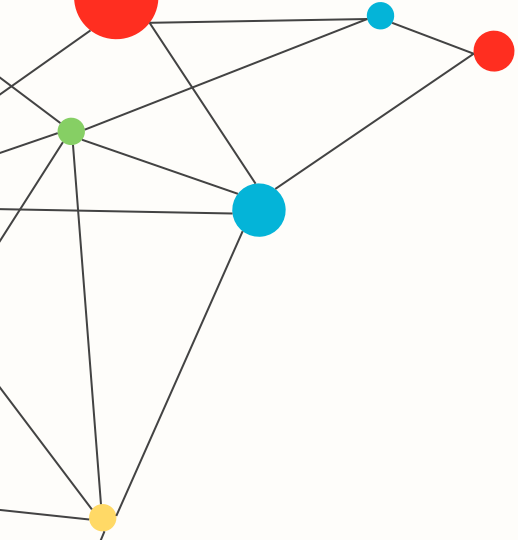
Burst Network Arrivals

Frames coming in bursts from the network can lead to throttling.

Sudden Decoder Stalls

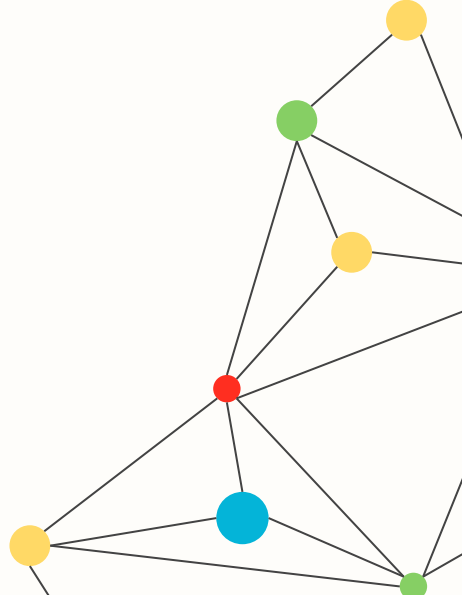
Failures in the decoder can lead to stalls





04

Evaluation



Experiment Setup

Real traces of Tencent's **START cloud gaming platform** were collected using the new system.

Results were compared with previous systems like **DropTail** (new packets are dropped if queue is full) and **FrameSkip** (skip processing or displaying frames if queue is full).



End-to-End Delay Improvement

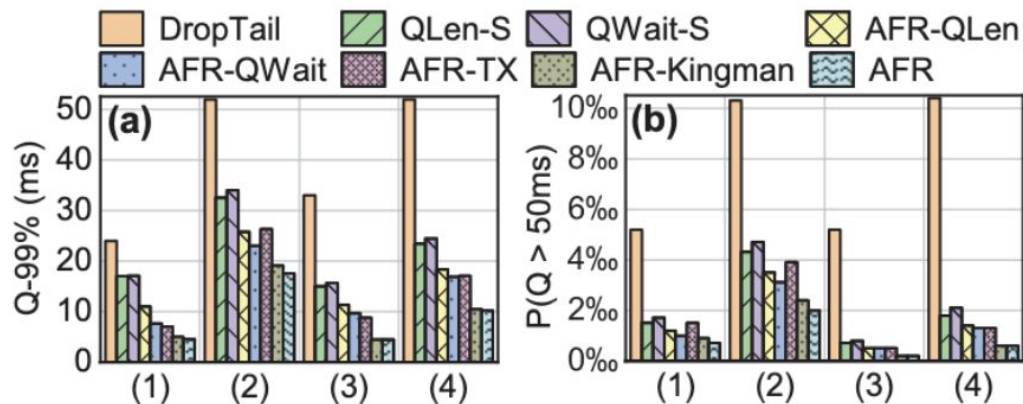
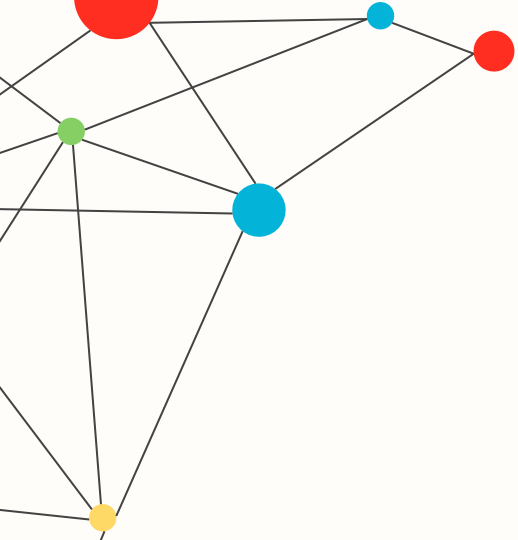


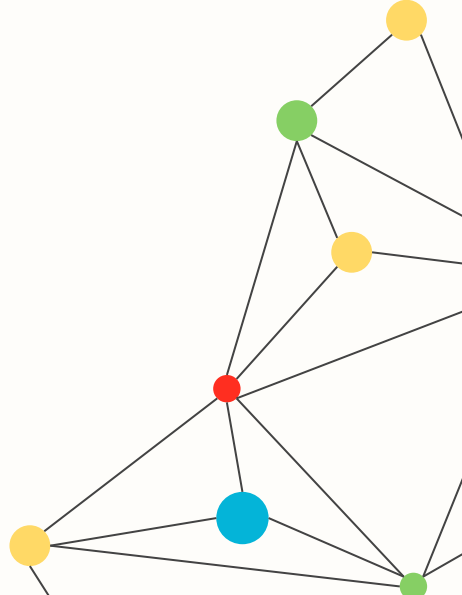
Figure 12: Simulation results of queuing delay (the 99%ile and the ratio of frames with >50ms queuing delay).

AFR improves the 99%ile total delay by 27% to 36%, and the ratio of severely delayed frames (total delay >100ms) by 1.6× to 2.2× in all traces



05

Takeaways



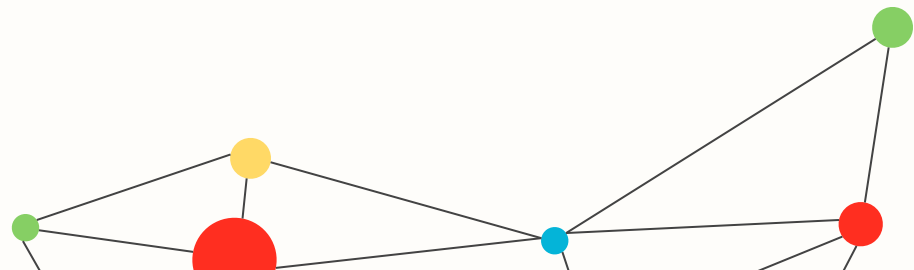


Takeaways

Client decoder queue is **overloaded** by increased video quality.

Frame rate is based on **arriving and departing frames** rather than monitoring queue length.

Adaptive Frame Rate was implemented in **34%** of Tencent's Cloud Gaming Platform.

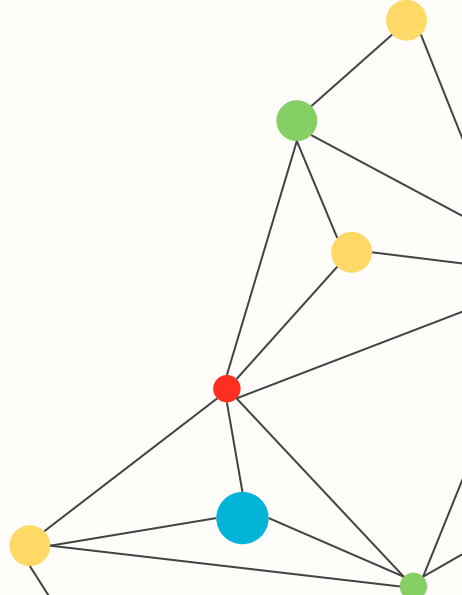
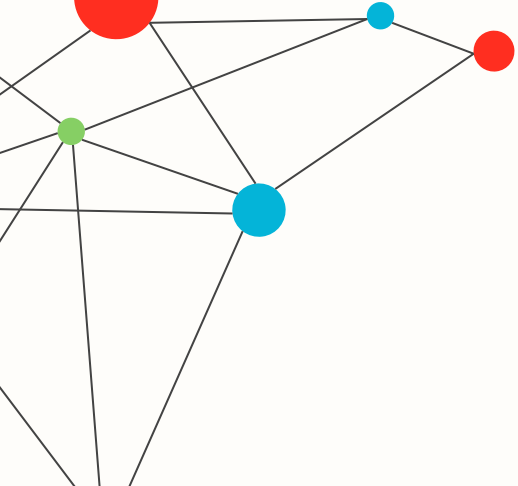


Future Research

Rather than live monitoring of the departure and arrival of frames in the queue, we could build a small **reinforcement learning model** to predict when we need to alter departure rates.

We can also test this framework for **lower resolution** and RTC as well as test for **traditional RTC using packets** instead of frames.





Thank you!